

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Технический дизайн микросервисной архитектуры

Выполнила:

Сачук А. А.

Группа:

БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Необходимо выполнить технический дизайн микросервисной архитектуры для backend-приложения рецептов. Нужно разделить монолитное приложение на отдельные микросервисы с учетом подхода database-per-service, описать взаимодействие между сервисами, разделить общую базу данных на независимые базы, а также спроектировать внутренние эндпоинты для межсервисного взаимодействия.

Ход работы

В качестве исходного приложения рассматривался сервис рецептов. В монолитной версии приложение включало регистрацию и авторизацию пользователей, создание рецептов, получение списка рецептов, фильтрацию по сложности и ингредиентам, просмотр деталей рецепта, создание комментариев и сохранение рецептов в избранное.

Для перехода к микросервисной архитектуре приложение было разделено по бизнес-областям. Каждый сервис отвечает за свою часть логики и имеет собственную базу данных. Такой подход снижает связанность между частями системы и позволяет развивать их независимо друг от друга.

Разделение приложения на микросервисы

Микросервис	Назначение	Основные данные
Auth Service	Регистрация, вход, проверка токена и получение данных пользователя.	users, refresh_tokens
Recipe Service	Создание рецептов, получение списка, фильтрация и просмотр деталей рецепта.	recipes, recipe_steps, recipe_ingredients
Comment Service	Создание и получение комментариев к рецептам.	comments
Favorite Service	Сохранение рецептов пользователем и получение списка избранного.	saved_recipes
Gateway API	Единая точка входа для клиента. Передаёт запросы в нужный сервис.	Собственной БД не имеет

В качестве основной схемы взаимодействия выбран API Gateway. Клиент отправляет запросы только в Gateway API, а он уже перенаправляет их в нужный микросервис. Например, запрос на создание рецепта направляется в Recipe Service, а запрос на добавление комментария - в Comment Service.

Общая схема архитектуры

```

Client
|
v
Gateway API
|-- Auth Service      -> auth_db
|-- Recipe Service    -> recipe_db
|-- Comment Service   -> comment_db
|-- Favorite Service  -> favorite_db

```

Сервисы взаимодействуют между собой через внутренние REST-запросы. Например, перед созданием комментария Comment Service может проверить существование рецепта через Recipe Service, а также получить данные пользователя через Auth Service. В дальнейшем часть взаимодействий можно вынести в очередь сообщений, например пересчет счетчиков комментариев или уведомления.

Разделение базы данных

База данных	Таблицы	Какой сервис использует
auth_db	users, refresh_tokens	Auth Service
recipe_db	recipes, recipe_steps, recipe_ingredients, ingredients	Recipe Service
comment_db	comments	Comment Service
favorite_db	saved_recipes	Favorite Service

Каждый микросервис работает только со своей базой данных. Например, Comment Service не обращается напрямую к таблице recipes. Если ему нужно проверить рецепт, он делает внутренний запрос в Recipe Service. Это соответствует принципу database-per-service.

Взаимодействие между микросервисами

Сценарий	Взаимодействие
Регистрация пользователя	Gateway API -> Auth Service
Вход пользователя	Gateway API -> Auth Service
Создание рецепта	Gateway API -> Auth Service для проверки токена, затем -> Recipe Service
Получение списка рецептов	Gateway API -> Recipe Service
Фильтрация рецептов	Gateway API -> Recipe Service с query-параметрами
Создание комментария	Gateway API -> Auth Service, Recipe Service, затем Comment Service
Сохранение рецепта	Gateway API -> Auth Service, Recipe Service, затем Favorite Service

Внутренние эндпоинты

Для межсервисного взаимодействия были выделены внутренние эндпоинты. Они не предназначены для прямого вызова с клиентской стороны и используются только между сервисами.

Метод	Эндпоинт	Назначение
GET	/internal/users/{userId}	Получить краткую информацию о пользователе.
POST	/internal/auth/validate	Проверить access token и вернуть id пользователя.
GET	/internal/recipes/{recipeId}	Проверить существование рецепта и получить его краткие данные.
GET	/internal/recipes/{recipeId}/owner	Получить владельца рецепта.
POST	/internal/recipes/{recipeId}/stats/comments/increment	Увеличить счетчик комментариев у рецепта.

Примеры запросов и ответов

Пример проверки токена пользователя:

```
POST /internal/auth/validate
Authorization: Bearer <access_token>
```

Response 200:

```
{
  "user_id": "11111111-1111-1111-1111-111111111111",
  "email": "user@mail.ru"
}
```

Пример проверки рецепта перед созданием комментария:

```
GET /internal/recipes/33333333-3333-3333-3333-333333333333
```

Response 200:

```
{
  "id": "33333333-3333-3333-3333-333333333333",
  "title": "Pasta with tomato and garlic",
  "author_id": "11111111-1111-1111-1111-111111111111"
}
```

Пример создания комментария через внешний API:

```
POST /api/v1/recipes/33333333-3333-3333-3333-333333333333/comments
Authorization: Bearer <access_token>
```

Request:

```
{
  "text": "Очень простой и вкусный рецепт"
}
```

Response 201:

```
{
```

```
"id": "44444444-4444-4444-4444-444444444444",  
"recipe_id": "33333333-3333-3333-3333-333333333333",  
"text": "Очень простой и вкусный рецепт",  
"created_at": "2026-06-08T09:30:00Z"  
}
```

Возможные ошибки

Код	Ошибка	Описание
400	bad_request	Некорректные параметры запроса или тело запроса.
401	unauthorized	Пользователь не авторизован или токен недействителен.
403	forbidden	У пользователя нет прав на выполнение действия.
404	not_found	Пользователь, рецепт или комментарий не найден.
409	conflict	Конфликт данных, например повторное сохранение рецепта.
500	internal_error	Внутренняя ошибка сервиса.

Шаги перехода от монолита к микросервисам

1. Выделить зоны ответственности внутри монолитного приложения.
2. Создать отдельные сервисы Auth, Recipe, Comment и Favorite.
3. Разделить общую базу данных на независимые базы данных по принципу database-per-service.
4. Перенести модели и логику каждого домена в соответствующий сервис.
5. Добавить Gateway API как единую точку входа для клиента.
6. Описать внутренние REST-эндпоинты для взаимодействия сервисов.
7. Проверить основные пользовательские сценарии: регистрация, вход, создание рецепта, комментарий и сохранение рецепта.

Вывод

В ходе выполнения работы был подготовлен технический дизайн микросервисной архитектуры для backend-приложения рецептов. Монолитное приложение было разделено на несколько сервисов: Auth Service, Recipe Service, Comment Service, Favorite Service и Gateway API. Для каждого сервиса была определена собственная зона ответственности и отдельная база данных.

Также были описаны способы взаимодействия между сервисами, внутренние эндпоинты, примеры запросов и ответов, а также возможные ошибки.

Полученная архитектура позволяет постепенно переходить от монолита к микросервисам.